BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

PERSONALIZED LEARNING RECOMMENDATION SYSTEM

A PROJECT REPORT

Submitted by

[STUDENT NAME(S)]

Roll Number(s): [SUMAN JANA]

Registration Number(s): [ENTER REGISTRATION NUMBER(S)]

Student Code(s): [BWU/BCA/24/502]

in partial fulfilment for the award of the subject

[MACHINE LEARNING] – [BCA57205(p)]

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

ACKNOWLEDGEMENT

We express our sincere gratitude to our project supervisor, [SUPERVISOR NAME], for providing valuable guidance, encouragement and technical suggestions throughout the development of the Personalized Learning Recommendation System. Their feedback helped us understand the project requirements, improve our approach and organize the work into manageable stages.

We are also thankful to the Head of the Department and the management of Brainware University for providing the academic environment, learning resources and facilities required to undertake this project.

Lastly, we extend our thanks to all group members for their cooperation and contribution. We are grateful to our parents, friends and classmates for their continuous encouragement and support during the completion of this academic work.

Project Group Members:

Student Name	Student Code	Physical Signature
SUMAN JANA	BWU/BCA/24/502	Suman jana

TABLE OF CONTENTS

ABSTRACT

LIST OF TABLES

LIST OF FIGURES

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

1. INTRODUCTION

  1.1 GENERAL

  1.2 PROJECT OVERVIEW AND BACKGROUND

  1.3 PROBLEM STATEMENT

  1.4 MOTIVATION

  1.5 SCOPE OF THE PROJECT

  1.6 LIMITATIONS

2. OBJECTIVES

  2.1 PRIMARY OBJECTIVE

  2.2 SPECIFIC OBJECTIVES

  2.3 EXPECTED OUTCOME

  2.4 PROJECT DELIVERABLES

3. PLANNING

  3.1 PROJECT DEVELOPMENT METHODOLOGY

  3.2 PROJECT TIMELINE

  3.3 WORK BREAKDOWN STRUCTURE

  3.4 TASK ALLOCATION

4. REQUIREMENT ANALYSIS

  4.1 HARDWARE REQUIREMENTS

  4.2 SOFTWARE REQUIREMENTS

  4.3 FUNCTIONAL REQUIREMENTS

  4.4 NON-FUNCTIONAL REQUIREMENTS

  4.5 FEASIBILITY STUDY

5. DATASET AND SYSTEM FLOW

  5.1 DATASET SELECTION AND DESCRIPTION

5.2 POSSIBLE DATASET FEATURES
5.3 DATA UNDERSTANDING
5.4 DATA CLEANING
5.5 DATA PREPROCESSING
5.6 EDA
5.7 FEATURE IDENTIFICATION
5.8 SYSTEM WORKFLOW
5.9 RECOMMENDATION APPROACH
5.10 FLOWCHART DESCRIPTION
6. PROPOSED DESIGN
6.1 SYSTEM ARCHITECTURE
6.2 USER INTERFACE DESIGN
6.3 DATA DESIGN AND DATA DICTIONARY
6.4 RECOMMENDATION OUTPUT DESIGN
6.5 MODULE DESCRIPTION
7. EXPERIMENTAL RESULTS AND EVALUATION
7.1 EXPERIMENTAL ENVIRONMENT
7.2 DATASET SUMMARY AND CLEANING RESULTS
7.3 EDA FINDINGS
7.4 RECOMMENDATION DEMONSTRATION
7.5 EVALUATION METHOD
7.6 TESTING
7.7 APPLICATION SCREENSHOTS
7.8 DISCUSSION OF RESULTS
8. FUTURE SCOPE
9. CONCLUSION
APPENDICES
REFERENCES
Note: Page numbers should be updated after final formatting.

ABSTRACT

The Personalized Learning Recommendation System is a machine-learning-based academic project intended to recommend suitable learning resources or topics according to individual student profiles. Students differ in their previous academic performance, subject interests, preferred learning formats, completed topics, study behaviour and learning history. A general list of educational materials may not address these differences. The proposed system aims to use student-related information to identify learning materials that may be relevant to an individual learner. The project workflow includes selecting or creating a suitable dataset, understanding and cleaning the data, preprocessing features, performing Exploratory Data Analysis, identifying important learning features, developing a recommendation approach, evaluating the recommendations and presenting the output through a simple interface. The expected prototype accepts student information and displays personalized learning topic or resource suggestions. The final implementation method, dataset details and measured results must be documented from the actual project work.

LIST OF TABLES

Table 3.1 Suggested Project Development Timeline
Table 4.1 Hardware Requirements
Table 4.2 Software Requirements
Table 5.1 Possible Student Profile and Resource Features
Table 7.1 Actual Dataset Summary
Table 7.2 Actual Recommendation Evaluation Results
Table 7.3 Functional Test Cases

LIST OF FIGURES

Figure 5.1 Proposed System Workflow
Figure 5.2 Data Cleaning and Preprocessing Workflow
Figure 5.3 Recommendation Process Flowchart
Figure 6.1 Proposed System Architecture
Figure 6.2 Student Input Form Wireframe
Figure 7.1 Dataset Preview
Figure 7.2 EDA Visualizations
Figure 7.3 Recommendation Output Screenshot

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

AI – Artificial Intelligence
CSV – Comma-Separated Values
EDA – Exploratory Data Analysis
ID – Identifier
ML – Machine Learning
MAE – Mean Absolute Error
RMSE – Root Mean Squared Error
UI – User Interface
UX – User Experience
LMS – Learning Management System
API – Application Programming Interface
K – Number of top recommendations considered in a ranking metric

CHAPTER 1 – INTRODUCTION

1.1 General

Education plays an important role in personal development, professional growth and the progress of society. In recent years, digital technology has changed the way students access educational materials. Students can now learn through online courses, recorded lectures, educational videos, digital books, articles, practice questions, quizzes and interactive learning platforms. These resources make education more accessible, but the large amount of available content can also create a new problem: students may find it difficult to decide what to study, which resource is appropriate for their current knowledge level and what topic they should learn next.

Every student has a different learning profile. Some students understand a concept more easily through video lessons, while others prefer written notes, books, examples or practical exercises. Students also differ in their academic performance, interests, completed topics, available study time and learning speed. Therefore, recommending the same material to every student may not always be effective. A resource that is useful for one student may be too easy, too difficult or less interesting for another student.

Machine Learning (ML) provides techniques that can analyse patterns in data and support personalized recommendations. A recommendation system uses available information about users and items to identify options that may be relevant to a particular user. In education, such a system can analyse student-related information and recommend learning topics or resources that match the student's learning needs.

The Personalized Learning Recommendation System is proposed to address this challenge. It aims to use student profile information and academic behaviour to suggest suitable learning resources or topics. The project combines data preparation, exploratory data analysis, feature identification, recommendation development, evaluation and a simple user interface. Its purpose is to demonstrate how machine-learning concepts can be applied to an educational problem.

1.2 Project Overview and Background

The project focuses on the development of a machine-learning-based Personalized Learning Recommendation System. The proposed application accepts information about a student and uses that information to identify suitable learning materials or topics. The student information may include previous academic performance, subject interests, preferred learning format, completed topics, study behaviour and learning history.

The system follows a data-driven workflow. First, a suitable student-learning dataset is selected or created. The dataset is then examined to understand its structure, features, data types and quality. Data cleaning and preprocessing are performed to handle missing values,

duplicate records, inconsistent categories and other data-quality issues. Exploratory Data Analysis (EDA) is conducted to identify patterns and relationships in the available information.

After the data has been prepared, important learning-related features are identified. A suitable recommendation approach is then selected according to the nature and quality of the dataset. Depending on the available data, the project may use content-based filtering, similarity-based recommendation, rule-based filtering or a suitable supervised machine-learning method. The selected approach produces recommendations based on the student's profile and the available learning-resource information.

The expected application is a simple working prototype. A student enters relevant information through an interface, submits the form and receives a list of recommended topics or resources. The recommendations should be displayed in a clear format so that the student can understand what to study next. The project is an academic prototype and should not be treated as a replacement for teachers, formal assessment or professional academic guidance.

1.3 Problem Statement

Students have access to a wide range of digital learning materials, but selecting the most suitable material is not always easy. General learning platforms may display popular or commonly used resources without fully considering each student's previous performance, interests, preferred learning format, completed topics and study behaviour. As a result, students may spend unnecessary time searching for resources, select content that does not match their current level or repeat topics they have already completed.

The main problem addressed by this project is the lack of simple, profile-based recommendations that help an individual student identify suitable learning topics or resources. The proposed system uses student-related information to generate personalized recommendations. It aims to make the learning-resource selection process more focused, relevant and convenient.

1.4 Motivation

The motivation for this project comes from the differences between individual learners and the increasing availability of online educational content. Students may require different explanations, formats and levels of difficulty to understand the same subject. A personalized recommendation system can help organize learning choices by considering relevant student information rather than showing identical suggestions to everyone.

The project also provides practical experience in applying machine-learning concepts to a real-world educational use case. It allows students to work with datasets, data preprocessing, EDA, feature analysis, recommendation techniques, evaluation metrics, Python libraries and user-interface development.

1.5 Scope of the Project

The scope includes selecting or preparing a student-learning dataset, understanding its features, cleaning and preprocessing the data, performing EDA, identifying important features, developing a recommendation approach, evaluating its output and presenting recommendations through a simple interface. The prototype may recommend topics, learning materials or resource formats depending on the information available in the dataset.

The initial version is limited to the dataset and features actually implemented by the project team. It does not guarantee improved examination marks or learning outcomes. Its recommendations are suggestions based on the available data and selected method. A larger real-world deployment would require broader data, user testing, privacy safeguards and continued evaluation.

1.6 Limitations

The quality of recommendations depends on the quality, size, relevance and representativeness of the dataset. A small or synthetic dataset may not represent the needs of all students. Some student preferences or learning behaviours may be difficult to measure accurately. If the dataset does not contain reliable information about whether a recommendation was useful, evaluation may be limited. The system may also face a cold-start problem when a new student has very little profile information. These limitations should be stated honestly in the final project report.

CHAPTER 2 – OBJECTIVES

2.1 Primary Objective

The primary objective of this project is to design and develop a simple machine-learning-based system that recommends suitable learning resources or topics to students according to their learning profiles and academic behaviour.

2.2 Specific Objectives

The detailed objectives of the project are as follows:

1. To identify the problem of selecting suitable learning topics and resources from a large collection of educational materials.

2. To select or create a suitable student-learning dataset containing relevant learning-profile information.

3. To inspect the dataset structure, feature names, data types, record count and data quality.

4. To perform data cleaning by identifying and handling missing values, duplicate records and inconsistent entries.

5. To perform data preprocessing, including encoding categorical values and scaling numerical features when required by the selected method.

6. To conduct Exploratory Data Analysis to understand student learning patterns and relationships among features.

7. To identify important features related to academic performance, interests, learning preferences, completed topics, study behaviour and learning history.

8. To select and implement an appropriate recommendation approach according to the dataset and project requirements.

9. To generate personalized recommendations for individual student profiles.

10. To evaluate the recommendation approach using suitable metrics or a documented user-feedback procedure.

11. To present the findings through tables, charts, graphs and application screenshots.

12. To develop a simple user interface through which students can enter their information and view recommended learning resources or topics.

2.3 Expected Outcome

The expected outcome is a simple working prototype that accepts student information as input and provides personalized learning recommendations. The output may include a recommended topic or resource title, subject/category, resource type and a short explanation

of why the recommendation may be relevant. The exact output fields will depend on the dataset and resource catalogue used in the implementation.

The project is also expected to produce a cleaned dataset, EDA visualizations, a documented recommendation method, evaluation results and a final project report. All numerical results and claims about model performance must be filled in only after the actual implementation and testing have been completed.

2.4 Project Deliverables

The main deliverables are: a selected or prepared dataset; a data-cleaning and preprocessing workflow; an EDA notebook or script; visualizations of important learning patterns; a recommendation method or trained model; a working input-and-output interface; test cases and evaluation results; application screenshots; source code; and the final project documentation.

CHAPTER 3 – PLANNING

3.1 Project Development Methodology

The project can be developed through an iterative and structured workflow. The work is divided into phases so that each stage produces an identifiable output. The team first defines the problem and requirements, then selects the dataset and studies its quality. After data preparation and EDA, the team selects features and develops the recommendation approach. The recommendation component is tested before it is connected to the user interface. Finally, the complete system is reviewed and documented.

An iterative approach is useful because the dataset may influence the choice of recommendation method. For example, if the dataset contains detailed resource attributes, content-based recommendation may be suitable. If it contains reliable labels or feedback, another method may be considered. The selected method should be justified by the actual data rather than chosen only because it is popular.

3.2 Project Timeline and Phase Breakdown

The following suggested schedule divides the work into eight weeks. The actual duration may be adjusted according to the university schedule and the number of group members.

Week 1 – Requirement Analysis: identify the problem, project objectives, expected input and output, scope and limitations.

Week 2 – Dataset Selection: search for a suitable dataset or prepare a documented sample dataset. Review the columns, data types, record count, source and permitted use.

Week 3 – Data Cleaning and Preprocessing: inspect missing values, remove duplicates where appropriate, standardize categories and prepare numerical and categorical features.

Week 4 – Exploratory Data Analysis: generate descriptive summaries and visualizations to understand the distribution of student-related features and identify useful patterns.

Week 5 – Feature Identification and Recommendation Design: select relevant features and compare possible recommendation approaches. Decide on the method that best fits the available data.

Week 6 – Model or Recommendation Development: implement the selected recommendation logic, prepare the resource catalogue and test recommendation generation with sample profiles.

Week 7 – Interface Development and Evaluation: build the input form and results page, connect the recommendation component and evaluate the system using the selected procedure.

Week 8 – Final Testing and Documentation: test different inputs, correct errors, capture screenshots, summarize findings and complete the project report and presentation.

3.3 Work Breakdown Structure

The project work can be divided into the following activities:

A. Requirement Analysis – define the user problem, project objectives, system boundaries, required inputs and expected output.

B. Dataset Preparation – identify a suitable dataset, verify its source and document its fields. If a dataset is created, explain how the records were prepared and whether they are synthetic.

C. Data Understanding – inspect the number of records, columns, data types, unique categories, missing values and basic statistical summaries.

D. Data Cleaning – handle missing data, duplicate rows, invalid values, inconsistent labels and irrelevant columns using documented decisions.

E. Data Preprocessing – encode categorical variables, scale numerical variables when needed and prepare consistent input transformations.

F. EDA – create charts and summaries to explore academic performance, interests, preferences, completed topics and study behaviour.

G. Feature Analysis – select the features that are relevant to the recommendation objective and explain why they are useful.

H. Recommendation Development – implement the chosen method and connect student profiles with the available topics or resources.

I. Evaluation – test the recommender using suitable metrics, test cases or user feedback and report actual results.

J. Interface and Integration – develop the student input form and recommendation display page and connect them with the recommendation logic.

K. Documentation – prepare diagrams, tables, code explanations, screenshots, references and the final report.

3.4 Task Allocation

For a group project, tasks may be allocated according to the number of members and their skills. One member may coordinate dataset selection and documentation; another may handle data cleaning and preprocessing; another may perform EDA and visualization; another may develop the recommendation logic; and another may build the interface and test integration. Every member should understand the complete project because the final evaluation may include questions about all modules. Replace these suggested responsibilities with the actual names and work allocation of the project group.

CHAPTER 4 – REQUIREMENT ANALYSIS

4.1 Hardware Requirements

The project can be developed on a standard personal computer. The exact hardware requirement depends on dataset size and the selected development environment.

Minimum suggested configuration:

• Processor: Intel Core i3 or equivalent dual-core processor.

• RAM: 4 GB minimum; 8 GB or more recommended.

• Storage: approximately 5 GB of available space for the development environment, dataset, source code and output files.

• Display: standard monitor or laptop display.

• Network: internet connection for installing libraries, accessing documentation or using an online notebook environment.

Recommended configuration:

• Processor: Intel Core i5 or equivalent.

• RAM: 8 GB or more.

• Storage: SSD with sufficient free space.

• Network: stable broadband connection.

These are practical suggestions for a small academic prototype, not fixed requirements for every machine-learning workload.

4.2 Software Requirements

Operating System: Windows 10/11, Linux or macOS.

Programming Language: Python 3.

Development Environment: Jupyter Notebook, Google Colab or Visual Studio Code.

Data Processing Libraries: Pandas and NumPy.

Data Visualization Libraries: Matplotlib and Seaborn.

Machine Learning Library: Scikit-learn, if its algorithms and utilities are suitable for the selected method.

Interface Framework: Streamlit or Flask may be used to create a simple application interface. The final report should name only the framework actually used.

File Format: CSV or another documented format for storing the dataset.

Web Browser: Google Chrome, Mozilla Firefox or another modern browser when a browser-based interface is used.

Version Control: Git may be used to track code changes and support collaboration.

4.3 Functional Requirements

1. The system should allow a student to enter relevant learning-profile information.

2. The system should accept the features supported by the selected dataset and recommendation method.

3. The system should validate required input values and provide a useful message when information is missing or invalid.

4. The system should preprocess the input using transformations consistent with those used during development.

5. The recommendation component should analyse the submitted profile and identify suitable topics or resources.

6. The system should display recommendations in an understandable format.

7. The system should handle cases where no suitable recommendation can be produced.

8. The system should allow another student profile to be submitted without restarting the complete application.

4.4 Non-Functional Requirements

Usability: the interface should be simple and understandable for students with basic computer knowledge.

Reliability: the application should produce consistent outputs for the same input when the underlying data and recommendation method have not changed.

Performance: recommendations should be generated within a reasonable time for the size of the selected dataset.

Maintainability: code should be divided into logical modules so that data preparation, recommendation logic and interface code can be updated separately.

Scalability: the design should allow additional resources, topics or student features to be added in later versions.

Privacy: personal information should be minimized, protected and used only for the stated academic purpose. Real student records should be anonymized where possible.

Transparency: the report should explain the recommendation method and its limitations. The system should not present recommendations as guaranteed academic outcomes.

4.5 Feasibility Study

Technical Feasibility: the project can be developed using commonly available Python libraries and a small interface framework. The selected method should be appropriate for the team's skills and the available dataset.

Operational Feasibility: the expected interface is simple: a student provides profile information and receives recommendations. Clear labels and instructions can make the prototype easy to use.

Economic Feasibility: the prototype can be developed using open-source libraries and free development tools. Any paid services or hosting requirements should be identified separately if they are introduced.

Time Feasibility: the project can be divided into manageable phases and completed within an academic schedule if the dataset and feature scope are kept realistic.

CHAPTER 5 – DATASET AND SYSTEM FLOW

5.1 Dataset Selection and Description

A student-learning dataset is the foundation of the recommendation system. The dataset should contain information that can help describe a student's learning profile and connect that profile to suitable topics or learning resources. The project may use a suitable public dataset or create a small dataset for academic demonstration.

Before using a dataset, the team should verify its source, permitted use, meaning of each column, number of records and limitations. If the dataset contains only academic performance and does not include learning resources, the team may need to create a separate resource catalogue or define a documented mapping between student features and available topics. The final report must clearly distinguish original collected data, public data and synthetic demonstration data.

The exact dataset name, download source, number of rows and number of columns are not specified in the project description. Therefore, these details must be completed after the team selects the actual dataset. No dataset statistics should be invented.

5.2 Possible Dataset Features

The following features are possible examples based on the project description. The final feature list must match the actual dataset.

Student_ID: an anonymized identifier used to distinguish records. Direct personal identifiers should not be used unless necessary and authorized.

Previous_Performance: a numerical mark, grade, score or performance category representing earlier academic achievement.

Interests: subject areas or topics in which the student has expressed interest.

Learning_Preference: the student's preferred format, such as video, written notes, articles, quizzes or practical exercises.

Completed_Topics: topics that the student has already studied or completed.

Study_Behaviour: information such as study frequency, time spent studying or learning activity, if available.

Learning_History: records of previously accessed, attempted or completed learning materials, if available.

Topic/Resource_ID: an identifier for an item in the resource catalogue.

Topic/Resource_Name: the title of the learning topic or material.

Resource_Type: the format of the material, such as video, notes, article, quiz or exercise.

Difficulty_Level: an available difficulty category, such as beginner, intermediate or advanced.

Subject_Category: the subject or topic area to which a resource belongs.

These fields are illustrative. A field should not be included in the final data dictionary unless it exists in the dataset or is deliberately added and documented by the project team.

5.3 Data Understanding

The first step after loading the dataset is to understand its structure. The team should inspect the first few records, the total number of rows and columns, the names of the columns and their data types. Numerical columns can be summarized using count, mean, standard deviation, minimum, maximum and quartiles. Categorical columns can be examined through unique values and frequency counts.

The team should also check whether the dataset contains a suitable link between student profiles and learning resources. A recommendation system requires a way to determine which resources may be relevant to a profile. If the dataset does not contain direct student-resource interactions, the team must explain how recommendations will be generated, for example through resource attributes, similarity calculations or documented rules.

5.4 Data Cleaning

Data cleaning improves the quality and consistency of the dataset before analysis. The following checks may be performed:

Missing Values: identify blank or null entries. Depending on the meaning of a field, missing values may be removed, replaced with a suitable statistic, assigned an explicit unknown category or handled by another documented method. The choice should be justified.

Duplicate Records: identify repeated rows or duplicate identifiers. Remove duplicates only when they represent repeated records rather than legitimate repeated learning activity.

Inconsistent Categories: standardize values that refer to the same category but use different spelling or capitalization. For example, category labels such as “Video”, “video” and “VIDEO” may need a consistent representation.

Invalid Values: check for impossible or out-of-range values, such as negative study time or marks outside the scale used by the dataset. Correct or remove such values only when the correct treatment is known.

Irrelevant Columns: remove columns that do not contribute to the project objective, while retaining fields needed for evaluation or traceability.

Outliers: examine unusual numerical values. An outlier should not automatically be deleted; first determine whether it is an error or a genuine observation.

Every cleaning decision should be recorded so that the process can be repeated and explained during project evaluation.

5.5 Data Preprocessing

Preprocessing transforms cleaned data into a form suitable for the selected recommendation method.

Categorical Encoding: text categories may need to be converted into numerical representations. One-hot encoding can be used for nominal categories, while ordinal encoding may be suitable when categories have a meaningful order. The encoding method should reflect the meaning of the data.

Numerical Scaling: numerical features measured on different scales may need standardization or normalization, particularly when a distance-based similarity method is used. For example, academic marks and study hours may have different numerical ranges.

Text Processing: if interests or completed topics are stored as text, the team may need to standardize text, split multiple values or represent topics using a suitable vectorization method.

Feature Preparation: create the final input columns required by the recommendation component. The same order and transformation rules must be used when processing new student input.

Train-Test Separation: if a supervised model is developed, separate training and testing data before fitting the model. Preprocessing steps that learn parameters from data should be fitted on training data and then applied to testing data to avoid data leakage.

The final report should describe the exact preprocessing operations actually performed.

5.6 Exploratory Data Analysis (EDA)

Exploratory Data Analysis is used to understand the dataset through summaries and visualizations. EDA helps identify distributions, unusual values, relationships and possible patterns that may influence recommendation design.

Suggested analyses include:

• Distribution of previous academic performance.

• Frequency of student interests or subject categories.

• Distribution of preferred learning formats.

• Frequency of completed topics.

• Study-time or study-frequency patterns, if recorded.

• Relationships between previous performance and study behaviour.

• Distribution of resource types and difficulty levels, if a resource catalogue is available.

• Counts of student-resource interactions, if interaction records exist.

Suggested visualizations:

1. Histogram – to show the distribution of numerical performance or study-time values.

2. Bar Chart – to compare the number of students in different interest or preference categories.

3. Box Plot – to examine the spread of marks or study time and identify possible outliers.

4. Correlation Heatmap – to inspect relationships among numerical features.

5. Count Plot – to compare categories such as learning preferences or resource types.

6. Stacked Bar Chart – to compare learning preferences across subject or performance groups, if appropriate.

Every graph in the final report should be generated from the actual dataset, have a clear title and axis labels, and include a short explanation of the observation. Do not claim that a chart proves a causal relationship when it only shows an association.

5.7 Feature Identification and Selection

Feature identification determines which information is useful for generating recommendations. Features should be selected according to the project objective, dataset quality and recommendation method.

Previous performance may help estimate an appropriate difficulty level. Interests may help identify relevant subject areas. Learning preferences may help choose the resource format. Completed topics may help avoid recommending material that the student has already finished. Study behaviour may provide information about learning frequency or available study time. Learning history may help identify previously used resources and topics.

Not every dataset will contain all these features. The team should select only the available and meaningful features. Features that are duplicated, unreliable, unrelated to the recommendation task or likely to expose unnecessary personal information should be excluded or carefully reviewed.

5.8 Proposed System Workflow

The proposed system follows the workflow below:

Step 1 – Student Input: the student enters the information requested by the application, such as academic performance, interests, preferred learning format, completed topics and study behaviour.

Step 2 – Input Validation: the system checks that required fields are provided and that values are within acceptable formats or ranges.

Step 3 – Input Preprocessing: the input is converted into the same representation used during dataset preparation. This may include category encoding, text processing or numerical scaling.

Step 4 – Profile Analysis: the recommendation component analyses the student profile using the selected recommendation method.

Step 5 – Resource Matching: available topics or resources are compared with the profile. Depending on the implemented approach, the system may apply similarity scores, filtering rules or model predictions.

Step 6 – Recommendation Selection: the system selects or ranks suitable items and may remove topics already marked as completed.

Step 7 – Output Display: the interface displays the recommended topics or resources, along with available information such as resource type, subject category or reason for selection.

Step 8 – Further Use: the student may change the profile information or submit another profile to receive an updated recommendation.

5.9 Recommendation Approach

The recommendation approach must be selected after examining the actual dataset. The following methods are possible options, not claims about what has already been implemented.

Content-Based Filtering: this approach recommends resources whose attributes match the student's profile or previously preferred content. Resource attributes may include subject, topic, difficulty and format. It can be useful when resource descriptions are available.

Similarity-Based Recommendation: a student profile can be represented as a feature vector and compared with other profiles or resource vectors using a similarity measure such as cosine similarity. The system can rank resources according to their similarity to the student's requirements. Feature scaling and encoding are important when using numerical similarity calculations.

Rule-Based Filtering: a small prototype may use clearly defined rules, such as recommending beginner resources when the student's profile indicates a beginner level, or filtering out topics already completed. This approach is easy to explain but may be less flexible than a data-driven recommender.

Collaborative Filtering: if the dataset contains enough student-resource interaction data, the system may identify patterns among students with similar interaction histories. This method is less suitable when interaction data is absent or extremely limited.

Supervised Machine Learning: if reliable target labels are available, a classification or regression model may predict an outcome related to resource suitability. The target variable must be clearly defined and supported by the dataset.

For a small academic project, a content-based or similarity-based method may be practical when the resource catalogue includes topic attributes. The final report must name the method actually implemented and explain the reasons for selecting it.

5.10 Flowchart Description

The main flowchart begins with Start, followed by student profile input. The system validates the information. If the input is invalid, an error message is displayed and the user is asked to correct the fields. If the input is valid, the system preprocesses the profile and sends it to the recommendation component. The component analyses the profile and selects suitable resources. The results are displayed to the student. The user may submit another profile or end the session. The flowchart should be drawn using standard symbols: oval for Start/End, parallelogram for input/output, rectangle for processing and diamond for decisions.

CHAPTER 6 – PROPOSED DESIGN

6.1 System Architecture

The proposed architecture is divided into four major components: the user interface, input-processing module, recommendation engine and recommendation-results module. The dataset and learning-resource catalogue provide the information required by the recommendation engine.

User Interface Layer: accepts student information through form fields and displays recommendations.

Input Processing Layer: validates submitted values and applies the preprocessing steps required by the recommendation method.

Recommendation Engine: analyses the processed student profile and matches it with available learning resources or topics.

Results Layer: formats and displays the selected recommendations in a readable manner.

Data Layer: stores or loads the student-learning dataset, resource catalogue and any model or preprocessing objects required by the application.

Separating these components makes the system easier to test and maintain. The interface can be changed without rewriting the recommendation logic, provided the input and output formats remain consistent.

6.2 User Interface Design

The proposed user interface should be simple, clean and suitable for students. The first screen may contain the project title and a short explanation of the system. A student profile form can then collect the fields supported by the dataset. Examples include academic performance, subject interest, preferred learning format, completed topics and study behaviour.

The form should use clear labels and helpful instructions. Dropdown lists can be used for fixed categories, while numerical fields can be used for marks or study time. Multiple selections may be used for interests or completed topics if the implementation supports them. A Submit or Get Recommendations button sends the information to the recommendation component.

The result page should show the recommended topics or resources in an organized list or cards. Each item may include a title, subject, resource type, difficulty level and reason for recommendation when those details are available. The interface should also provide a way to return to the form and submit another profile.

Actual screenshots of the completed application must be inserted in the final report. A proposed wireframe should be labelled as a design, not as a screenshot of a working application.

6.3 Data Design and Data Dictionary

A data dictionary explains the fields used by the project. The following table is a suggested structure and must be adjusted to match the actual dataset and resource catalogue.

Student profile fields:

• student_id – anonymized student record identifier.

• previous_performance – earlier academic score or category.

• interests – one or more subject/topic interests.

• learning_preference – preferred learning-resource format.

• completed_topics – topics already studied by the student.

• study_behaviour – available study-frequency or study-time information.

• learning_history – available record of earlier learning activity.

Resource catalogue fields:

• resource_id – unique identifier for a learning resource.

• resource_name – title of the learning material.

• subject_category – subject or topic group.

• resource_type – video, notes, article, quiz or another format.

• difficulty_level – difficulty category, if available.

• topic_tags – keywords describing the content.

These fields are examples. Data types, constraints and required fields must be documented from the actual implementation. If the system stores student information, avoid unnecessary personal details and apply suitable access controls.

6.4 Recommendation Output Design

The output should help the student understand the next learning steps. A recommendation list may display the resource or topic name, subject category, resource type and difficulty level. If the algorithm calculates a similarity or relevance score, the interface may show it only if its meaning is explained and it is not misleadingly presented as a probability of success.

A short explanation can improve transparency. For example, a system may state that a resource matches the student's selected subject interest or preferred learning format. The explanation must be based on the actual recommendation logic and available fields. The

application should not claim that a recommendation is guaranteed to improve marks or learning performance.

6.5 Module Description

Module 1 – Dataset Loading: reads the selected dataset from its file or storage location.

Module 2 – Data Cleaning: handles missing values, duplicate records and inconsistent entries.

Module 3 – Data Preprocessing: transforms numerical, categorical or text data into a suitable representation.

Module 4 – EDA and Visualization: produces descriptive summaries and graphs for analysis.

Module 5 – Feature Selection: identifies the input fields used by the recommendation approach.

Module 6 – Recommendation Engine: matches a student profile with topics or resources using the selected method.

Module 7 – User Interface: accepts student input and displays the recommendation results.

Module 8 – Evaluation: tests the recommendation process and records the actual performance or feedback.

CHAPTER 7 – EXPERIMENTAL RESULTS AND EVALUATION

7.1 Experimental Environment

This section should describe the actual environment used to run the project. Include the Python version, operating system, IDE or notebook, libraries and interface framework. Mention the computer configuration only if required by the project guidelines. The report should identify the versions used where possible so that the experiment can be reproduced.

7.2 Dataset Summary and Cleaning Results

Insert the actual dataset name, source, total number of records, number of columns and a short description of the important features. Report the number of missing values and duplicate records found before cleaning, the actions taken and the resulting dataset size. If the dataset was created for demonstration, clearly state that it is synthetic and explain how its records were generated.

Suggested table:

Dataset Name: [ENTER ACTUAL DATASET NAME]

Source: [ENTER SOURCE OR STATE THAT IT WAS CREATED]

Initial Records: [ENTER ACTUAL COUNT]

Final Records: [ENTER ACTUAL COUNT]

Number of Features: [ENTER ACTUAL COUNT]

Missing Values: [REPORT ACTUAL FINDINGS]

Duplicate Records: [REPORT ACTUAL FINDINGS]

Do not leave fabricated numbers in this section. Replace every bracketed field after running the actual data-preparation code.

7.3 EDA Findings and Visualizations

Insert the graphs generated during EDA. For every graph, provide a figure number, title and a short interpretation. Examples include the distribution of academic performance, the frequency of subject interests, the distribution of preferred learning formats and a comparison of study behaviour across performance groups.

The explanation should describe what is visible in the graph. For example, it may identify the most frequent category or describe whether the values are concentrated in a particular range. Avoid making unsupported claims about why a pattern occurs. If the dataset is small

or synthetic, mention that the observations are limited and may not generalize to a wider student population.

7.4 Recommendation Demonstration

Demonstrate the system using a few test profiles. Each test case should include the input values and the recommendations produced by the application. Use anonymized or synthetic profiles unless there is explicit permission to use real student information.

Test Case 1: [DESCRIBE PROFILE 1 AND INSERT ACTUAL RECOMMENDATIONS]

Test Case 2: [DESCRIBE PROFILE 2 AND INSERT ACTUAL RECOMMENDATIONS]

Test Case 3: [DESCRIBE PROFILE 3 AND INSERT ACTUAL RECOMMENDATIONS]

Explain why the system selected the displayed resources according to the implemented logic. If the recommendations are generated using rules or similarity scores, describe the relevant rule or matching features. Do not invent outputs before running the application.

7.5 Evaluation Method

The evaluation method should match the type of recommendation system implemented.

For a ranked recommender with relevant-item labels, Precision@K can measure the proportion of the top K recommended items that are relevant. Recall@K can measure the proportion of all relevant items that appear in the top K recommendations. Coverage can describe how much of the available resource catalogue can be recommended.

If explicit relevance labels are unavailable, a small user-feedback study may ask participants to rate whether recommendations are relevant, understandable and appropriate. The number of participants, questions and response scale should be reported. Feedback from a small academic demonstration should not be generalized to all students.

If the project instead predicts a defined label using a supervised model, use suitable classification or regression metrics. For classification, metrics may include accuracy, precision, recall and F1-score. For regression, metrics may include MAE or RMSE. These metrics should be used only when the project actually has a defined prediction target and an appropriate test procedure.

A recommendation score or model accuracy should never be invented. Report the actual evaluation procedure and results, including limitations.

7.6 Testing

Functional testing should verify that the application accepts valid input, detects invalid input, applies preprocessing consistently, generates recommendations and displays results correctly. Test profiles should include different combinations of interests, performance levels, preferences and completed topics, according to the fields supported by the application.

Example test cases:

1. Valid student profile – the system should accept the input and display recommendations.

2. Missing required field – the system should display a validation message.

3. Invalid numerical value – the system should reject or request correction of the value.

4. Previously completed topic – the system should avoid recommending it again if the implementation includes this filtering rule.

5. No matching resource – the system should display a suitable message rather than failing.

6. Second profile submission – the system should process a new profile correctly.

Record the actual result for each test case and mark it as Pass or Fail after execution.

7.7 Application Screenshots

Insert screenshots captured from the actual working application. Recommended screenshots include the dataset preview, missing-value or cleaning summary, EDA charts, recommendation logic output, student input form and final recommendation results page. Every screenshot should have a figure number and a descriptive caption. Ensure that screenshots do not expose private student information, passwords or API keys.

7.8 Discussion of Results

Discuss whether the system meets the stated project objectives. Explain what the dataset revealed, which features were used, how recommendations were generated and what the evaluation indicates. Mention cases where recommendations were useful and cases where the system produced limited or unsuitable results. Discuss the effect of dataset size, missing information, limited resource descriptions and lack of feedback if these issues apply.

The discussion should be based on actual experiments. If the prototype demonstrates the workflow but has not been evaluated with real students, state that it is a proof of concept and identify user evaluation as future work.

CHAPTER 8 – FUTURE SCOPE

8.1 Scalability and Feature Extensions

The Personalized Learning Recommendation System can be improved in several ways. A larger and more diverse dataset could help the system represent a wider range of student profiles and learning needs. The resource catalogue could be expanded to include more subjects, topics, difficulty levels and resource formats.

A future version may collect optional feedback after a student uses a recommendation. This feedback could help identify whether a resource was relevant, understandable and useful. Recommendations could then be updated as the student's learning history changes. A progress dashboard could show completed topics, pending topics and suggested next steps.

The system could also integrate with a Learning Management System (LMS) to use authorized learning activity data. More advanced recommendation methods, including hybrid approaches that combine content-based and collaborative filtering, may be explored if sufficient interaction data becomes available. Natural language processing could be used to analyse topic descriptions or learning-resource text.

Further improvements may include multilingual resources, accessibility features, mobile-friendly design, explainable recommendations and teacher or administrator dashboards. Any expansion involving personal learning data should include appropriate privacy, security, consent and data-retention practices.

CHAPTER 9 – CONCLUSION

9.1 Project Summary

The Personalized Learning Recommendation System is designed to address the difficulty students may face when selecting suitable learning resources from a large collection of educational materials. Students have different academic backgrounds, interests, learning preferences, completed topics and study behaviours. A general recommendation list may not address these differences effectively.

The proposed project applies machine-learning and recommendation-system concepts to student learning information. Its workflow includes dataset selection or creation, data understanding, data cleaning, preprocessing, exploratory data analysis, feature identification, recommendation development, evaluation and user-interface design. The expected prototype accepts a student's profile and displays relevant learning topics or resources.

The project provides practical experience in data analysis, Python programming, machine-learning workflows, visualization, recommendation logic and application development. The final success of the project should be assessed using the actual implementation and evaluation results. The final submitted conclusion must therefore be updated to state the dataset used, the method implemented, the results obtained and the limitations observed. The system should be presented as a support tool that can assist students in choosing learning materials, not as a replacement for teachers or formal academic judgement.

APPENDICES

Appendix A – Dataset Information

Include the dataset name, source, access date, licence or usage conditions, number of records, number of features and column descriptions. If a dataset is created, describe how records were generated and clearly label them as synthetic or demonstration data. Include any anonymization or privacy steps.

Appendix B – Important Source Code

Include selected code segments for loading the dataset, inspecting data, handling missing values, preprocessing features, generating EDA graphs, calculating recommendations and running the interface. Add a short explanation before or after each code segment. The complete source code may be submitted separately if required by the university. Never include passwords, API keys or private student records.

Appendix C – Additional Graphs and Screenshots

Add additional EDA graphs, system workflow diagrams, interface wireframes, test-case screenshots and recommendation output screenshots that support the main chapters. Each item should have a figure number and caption.

Appendix D – Sample Test Case Format

Test Case ID: [ENTER ID]

Test Description: [ENTER DESCRIPTION]

Input: [ENTER TEST INPUT]

Expected Output: [ENTER EXPECTED OUTPUT]

Actual Output: [ENTER ACTUAL OUTPUT]

Status: [PASS/FAIL]

Remarks: [ENTER REMARKS]

REFERENCES

References and Technical Documentation

Use the references that were actually consulted during the project. The following are suitable starting points for the technologies mentioned in this report; add the exact dataset citation and any additional books or articles used.

1. Pedregosa, F. et al. (2011). “Scikit-learn: Machine Learning in Python.” Journal of Machine Learning Research, 12, pp. 2825–2830.

2. McKinney, W. (2010). “Data Structures for Statistical Computing in Python.” Proceedings of the 9th Python in Science Conference.

3. pandas documentation. Available at: https://pandas.pydata.org/docs/

4. NumPy documentation. Available at: https://numpy.org/doc/

5. Matplotlib documentation. Available at: https://matplotlib.org/stable/

6. Seaborn documentation. Available at: https://seaborn.pydata.org/

7. Scikit-learn documentation. Available at: https://scikit-learn.org/stable/

8. Streamlit documentation, if Streamlit is used. Available at: https://docs.streamlit.io/

9. Flask documentation, if Flask is used. Available at: https://flask.palletsprojects.com/

10. [ADD THE ACTUAL DATASET REFERENCE, AUTHOR/ORGANIZATION, URL AND ACCESS DATE.]

Check the bibliographic details and access dates before final submission. Do not list a tool or source as a reference unless it was used or consulted.